



RESOURCES · REFERENCE IMPLEMENTATION

Hermes at Riverside. *An open-source substation copilot.*

I put my money where my mouth is. Hermes, the same agent runtime I run in production, wired up as a shadow-mode copilot for a hypothetical utility substation. Grounded in public synthetic data. Free for the world to run. Built exactly the way the articles say it should be built, right down to the self-improvement loop.

ADAM BROWN
AUTHOR

9 MIN
READING TIME

APR 2026
PUBLISHED

CASE STUDY
REFERENCE

SOURCE ARTICLES

Part 1

Shadow AI →

Part 2

Regulatory landscape →

Part 3

Three-zone architecture →

Part 4

Implementation →

Essay

The Agentic Epoch →

I write about autonomous substation operations. I argue that the stack exists today, that the governance is tractable, that open-weights models running inside the fence can reason about real operations problems, and that utilities who wait for a vendor pitch deck will buy the same thing two years late and at four times the price. At some point that kind of writing has to earn its keep. So here's a working copilot.

Hermes is the agent runtime I named in [The Agentic Epoch](#). It's the one I actually run: persistent Obsidian-vault memory, sub-agents that spawn per task and retire when done, MCP tools into everything worth reaching, nightly consolidation agents that review recent decisions and surface contradictions. I run it in an office closet today. This repo is Hermes wired up for a substation, answering questions against the [Dynamic Network Model](#), a public synthetic 238K-customer utility I published earlier this year. It reasons about voltage optimization, DER absorption, and restoration switching on a specific substation. It uses Gemma 4 E4B running on commodity hardware. It improves itself while it runs. The repo is Apache-2.0. Any utility can clone it tonight and have it answering questions about their own feeders tomorrow.

THE COMMITMENT

Apache-2.0 on every line. No sign-up. No vendor lock-in. No proprietary inference path. Swap the data adapter, point at your own GIS, OMS, historian, and interconnection tracker, and the agent is yours. If your security team wants the Bedrock-over-VPC path instead of local open weights, the compliance gate ships in the same repo.

Open source, autonomous substation operations, free for the world to use. That's the whole pitch.

What Hermes is doing here

Hermes is a shadow-mode copilot for a substation operator. It runs on a commodity industrial PC inside the fence. It answers questions the operator used to carry around in their head. It never actuates. Every recommendation is a draft for human review, per [Rung 2 on the autonomy ladder](#).

The runtime is the same one that powers my personal agent stack at home: Obsidian-vault memory, sub-agents per task, MCP-gated tools, nightly consolidation. The only thing that changes when you point Hermes at a substation is the tool surface and the data it reasons over. The same runtime pattern that reminds me about a dentist appointment is the one drafting a VVO recommendation on Riverside's west feeder, because the runtime doesn't care what the task is.

Event-driven, not operator-prompted

Hermes doesn't wait for an operator to ask a question. Every scenario in the panel below is dispatched by a detection event from the utility's existing telemetry. Power-quality flags (upper-band voltage, fast dV/dt) fire the VVO scenarios. AMI last-gasp clusters fire the restoration scenarios, before the OMS has finished opening its ticket. A microgrid controller's grid-tie-loss signal routes the restoration run to the islanding branch.

The operator's role isn't to hand-draft the query. The operator confirms. Every recommendation lands in front of them with the trigger that fired it, the tools Hermes called, and the data those tools returned.

Identity and playbooks live in one markdown file

The trigger layer doesn't hardcode what Hermes does on each event. It emits a detection signal with a `playbook_key` and a context dictionary. The agent then reads its identity and its per-event playbook from a single file:

```
hermes/agent/HERMES.md .
```

That markdown file is the only knob a utility turns to customize Hermes. It carries the identity (Rung 2 posture, CEII handling, citation discipline) and five playbooks keyed by event kind (`upper_band_voltage` , `far_end_undervoltage` , `dvd_t_storm` , `ami_last_gasp_cluster` , `microgrid_islanding`). Each playbook is a template that gets rendered with the event's context and handed to the agent as the first user turn.

HERMES.MD · AMI_LAST_GASP_CLUSTER PLAYBOOK

```
### ami_last_gasp_cluster
```

```
Trigger: {source} reports {count} AMI last-gasp messages on {feeder_id} in  
the last {window_seconds} seconds. OMS ticket is opening. Timestamp  
{timestamp}.
```

```
Draft a restoration plan. Use the topology (sectionalizers, open points, tie  
switches) to isolate the faulted section and restore as many customers as the  
topology permits via adjacent feeders. Call out what requires operator  
judgment.
```

Three knobs in increasing order of invasiveness: rewrite the playbook templates (no code change), add a new playbook and wire a detection threshold for it in `hermes/triggers.py` , or rewrite the identity section to shift the agent's posture up or down the autonomy ladder. The watcher layer itself is about 80 lines of Python — a polling pattern any utility can wire into its historian, AMI head-end, and OMS adapter.

Memory is append-only and nightly-curated

Hermes doesn't forget. Every turn — the trigger that fired, the tools called, the data returned, the recommendation produced, and the operator's response to it — goes into an append-only event log scoped to this substation. That log is the raw material the agent learns from.

A consolidation sub-agent wakes up every night at 22:00 and does four things: it dedupes repeated dispatches for the same sustained event, surfaces contradictions where today's recommendation drifted from what Hermes shipped a week ago on a similar event, rolls per-feeder voltage and load baselines forward with today's data, and tags every turn where the operator overrode or edited Hermes so those examples feed the next autoresearch iteration. This is the same consolidation pattern that runs in the production Hermes stack; nothing about it is substation-specific.

The event log is CEII. Aggregates only, no individual customer records. On-prem storage. Audit trail ships to the utility SIEM per CIP-007. Cross-substation learning is out of scope at Rung 2 — each substation's Hermes runs with its own log and its own consolidation agent. Coordinated reasoning across substations is a Rung 5 concern, well past the horizon of this example.

Porting Hermes to a second substation

The portability claim is that a utility customizes Hermes by editing one markdown file and the rest of the stack doesn't move. To show the claim with numbers, the repo ships a second identity file: `HERMES-chandler-heights.md`, a port to SP&L's Chandler Heights substation (SUB-014).

Chandler Heights is a different character of substation: 69/12.47 kV, six feeders instead of three, 87% utilization, triple Riverside's DER density (1,528 solar installs, 69.5 MW), and an outage archive dominated by equipment

failure rather than weather. It hosts a university microgrid (GCU North Campus, 2 MW solar + 1.5 MW / 6 MWh battery, 3-hour island) with a different critical-load profile than Luke AFB.

The diff: the entire Identity section swaps for Chandler's facts. Three playbooks (`upper_band_voltage` , `far_end_undervoltage` , `dvd_t_storm`) are byte-for-byte unchanged. Two playbooks get tuned:

`ami_last_gasp_cluster` adds a cue to check preventive-maintenance history first because that's the dominant failure mode;

`microgrid_islanding` references GCU's longer battery runway and mixed university critical load. That's the whole port. Same triggers, same tools, same agent loop, same rendering layer. Deploy, run, repeat.

In this reference implementation the agent wears two hats:

- **VVO advisor.** Given a timestamp and a feeder, recommend cap-bank switching and LTC tap setpoints that hold voltage inside the band while absorbing DER generation. Reason about the hosting-capacity limit. Coordinate with BESS on the evening peak.
- **Restoration planner.** Given an outage record, draft a switching sequence using the available sectionalizers, open points, and tie switches. On the feeder that hosts a microgrid, reason about when to island, when to resync, and what happens if the grid is still down at the end of the battery runtime.

Both hats are drawn directly from the use cases [named in the Agentic Epoch essay](#). They're the two that map cleanest to shadow-mode operation on a distribution substation with a real DER population.

The substation

The demo runs against **Riverside (SUB-001)**, a 230/12.47 kV substation in the west Valley of the Dynamic Network Model's synthetic Phoenix utility. I chose it for four reasons: three feeders is enough topology to tell a restoration story without drowning the narrative; one of the three feeders hosts a microgrid, which unlocks the islanding scenario; the substation is running at 82% of rated capacity, which makes the VVO headroom argument tactile; and the outage history carries 117 events, enough variety to pull a clean single-feeder restoration scenario and a messier multi-feeder one.

Substation

SUB-001 · Riverside · 230/12.47 kV

Rated / peak

20 MVA / 16.5 MVA (82% utilization)

Feeders

FDR-0001 (W), FDR-0002 (N), FDR-0003 (S)

Microgrid

Luke AFB Annex on FDR-0002 · 4 MW solar + 3 MW / 12 MWh BESS + 2.5 MW CHP · 2.3 h island

DER

645 solar installs · 24.2 MW installed · 147 BESS sites

Hosting capacity

48.9 MW binding · voltage-limited on 766/807 transformers

Outage history

117 logged events across 2020–2024

Every number in that block was pulled live from the Dynamic Network Model at page render by the demo repo's SP&L adapter. None of it was hand-tuned for the story. The voltage-hosting ratio is the one that matters most. 95% of transformers at Riverside hit their voltage limit before their thermal

limit, which means reactive support, not reconductor, is the near-term lever. That's the kind of observation a VVO advisor earns its keep making.

§ 03

Five scenarios, replayed

The panel below is an animated replay of the agent running through five scenarios against Riverside. Three are VVO, two are restoration. Each trace was recorded once against Gemma 4 E4B on commodity hardware, captured as JSON, and replayed here with the timing compressed for reading. The tool calls and the final text are exactly what the model produced. Expand any tool card to see the data the agent saw.

HERMES · RIVERSIDE COPILOT

VVO ADVISOR

OLLAMA · GEMMA4:E4B · 3 TOOLS

§ 01 AFTERNOON DER ABSORPTION

§ 02 EVENING VOLTAGE SAG

§ 03 MONSOON STORM – DER VS VOLTAGE

§ 04 SINGLE-FEEDER RESTORATION

§ 05 MICROGRID ISLANDING

Riverside's west feeder (FDR-0001) holds the highest concentration of rooftop solar on the substation. On a clear July afternoon the feeder runs near voltage ceiling as PV peaks. Hermes self-dispatches when the power-quality monitor flags upper-band voltage at multiple AMI endpoints.

POWER QUALITY · AMI EVENT

V > 1.052 pu at 12 of 47 sampled AMI meters on FDR-0001 · PF drifting · solar ramp detected

↳ Hermes dispatched: 01-afternoon-der (VVO advisor)

HERMES (SELF-PROMPT)

It's July 19, 2023, 2pm local. FDR-0001 is carrying heavy solar output and voltage is pushing the upper band. What should I do for the next 15

minutes? Keep it specific — name the devices and the direction of change.

get_load_snapshot {"timestamp":"2023-07-19 14:00"}

EXPAND

get_solar_snapshot {"timestamp":"2023-07-19 14:00"}

EXPAND

get_hosting_capacity {}

EXPAND

HERMES

Based on the data for SUB-001 at 14:00 on 20

SCENARIO	
Id	01

Role	VVO advisor
TRACE	

Provider	ollama

Model	gemma4:e4b

Tool calls	3

Wall time	77.3s
RIVERSIDE	

Substation	SUB-001

HV / LV	230 / 12.47 kV

Feeders	3

Microgrid	Luke AFB Annex

Hosting cap	48.9 MW

Outages	117

REPLAY

SKIP TO END

SPEED

1×

Recorded trace, replayed for clarity. Hermes ran once against Gemma 4 E4B on commodity hardware; this animation replays the exact tool calls and final response from that run. Reasoning time has been compressed for

reading.

What each scenario is actually demonstrating

Scenarios 1–3 walk the VVO story from clean July afternoon (DER at peak, voltage pushing the upper band) to evening sag (solar gone, load still up) to monsoon storm (fast solar transients, operator needs a ride-through posture). The interesting move across the three is that Hermes doesn't reach for the same lever every time. Afternoon absorption wants reactive support from the existing DER fleet. Evening sag wants BESS dispatch. The storm wants the agent to explicitly say *stop chasing DER and hold the voltage*.

Scenarios 4–5 demonstrate the restoration work. Scenario 4 is a single-feeder weather outage on FDR-0003: the agent pulls the outage record, pulls the topology, and drafts a sequence that uses the available sectionalizers and open points to restore as many customers as the topology allows while the faulted section is isolated. Scenario 5 is the microgrid islanding case on FDR-0002. This one sits at the intersection of protection coordination, battery state-of-charge, and the customer the utility cannot afford to drop. The agent reasons about the 2.3-hour battery window, the 2.5 MW CHP that can extend it, and what happens if the grid is still out when both run down.

§ 04

The governance that makes this shippable

The hard part of deploying an agent inside the fence was never getting the model to answer the question. It was making sure model inference can't cross a boundary the utility hasn't approved. Substations carry CEII. Utilities operate under NERC-CIP. A SaaS LLM call is not allowed, will not be allowed,

and shouldn't be allowed. So the repo ships with the compliance boundary built into the code path.

The default inference provider is local Ollama. The second, third, and fourth providers (vLLM, llama.cpp, Bedrock-over-VPC) are selectable via environment variables. The Bedrock path is gated. Selecting it without a VPC-endpoint attestation aborts startup.

HERMES/CONFIG.PY · COMPLIANCE GATE

```
$ HERMES_LLM_PROVIDER=bedrock python -m hermes.cli chat
```

```
[hermes] compliance gate blocked startup:
```

```
Bedrock provider requires HERMES_BEDROCK_VPC_CONFIRMED=1.
```

```
Set this only after your utility's security team has confirmed
```

```
a VPC endpoint path to Bedrock and approved CEII data flowing over it.
```

```
See docs/SECURITY.md.
```

Pointing at a public `bedrock-runtime.us-east-1.amazonaws.com` endpoint with the flag set also aborts. The gate enforces that

`AWS_ENDPOINT_URL_BEDROCK` resolves to a `*.vpce.amazonaws.com` host.

AWS PrivateLink or nothing. This is the artifact the repo hands your security team. It's not a warning in a README. It's control-flow in `hermes/config.py`.

WHY IT MATTERS

The [three-zone architecture](#) only works if the boundaries are enforced in code, not in policy. A governance posture that lives in a Confluence page loses to a developer in a hurry. A gate that lives in the inference adapter never loses, because it fires on every process start.

The stack, for the utility reader

Everything in the repo is Apache-2.0 or permissively licensed. The only commercial component is the optional Bedrock path, and that one is only used if the utility has approved it. Production deployments need no third-party licenses.

Agent runtime

Hermes (Obsidian-vault memory, sub-agents, MCP tool gateway, nightly consolidation)

Model

Gemma 4 E4B (open weights, MoE, edge-optimized)

Inference host

Ollama (default) · vLLM · llama.cpp · Bedrock-VPC (gated)

Adapter

LiteLLM (single call site, provider-agnostic)

Agent loop

Hand-rolled tool calling. No LangChain. 200 lines, auditable.

Self-improvement

Autoresearch loop scores the agent against the eval set, proposes prompt edits, keeps improvements, discards regressions.

Data

SP&L 23-dataset synthetic (published, Apache-2.0)

Embeddings

bge-small-en · LanceDB · local file-backed vector store

UI

Jupyter notebook (static) · Streamlit (live + replay modes)

Docs

SECURITY.md · DEPLOYMENT.md · per-zone deploy recipes

Swap the SP&L adapter for the utility's systems of record. The five scenarios in the panel above draw from four of them: GIS for the one-line and

nameplate (substation, feeders, transformers, breakers, sectionalizers, open points), **OMS** for the outage archive that the restoration scenarios read, **historian** for the 15-minute load and voltage snapshots, and the **interconnection tracker** for DER and microgrid inventory. Weather is a direct service call. The rest of the stack doesn't move. That's the contract: `hermes/data/*.py` is the only place that reads files. Everything above it is provider-agnostic, schema-stable, and doesn't care where the data came from.

CMMS shows up later, not here. A predictive-maintenance scenario (DGA flag on a transformer, thermal-scan delta on a bushing, open a work order) is a natural next tool surface. It's deliberately out of scope for this round — the five scenarios are scoped to VVO and restoration so the narrative stays tight.

§ 06

Self-improvement in the same loop

The agent and the evaluator share a repo. That's the part of the architecture I care about most, because it means Hermes can tighten its own prompt without a human rewriting it.

The pattern is [Karpathy's autoresearch](#) applied to an operations copilot. Define yes/no evaluation criteria. Run the agent against the same five scenarios. Score each response. Propose one prompt edit. Re-score. Keep if the score went up, discard if it went down. Iterate. The criteria are tuned to what a substation operator actually wants: asset IDs get cited, responses stay under 200 words, tools get called before reasoning, shadow-mode language reads like a draft instead of a command.

The panel below walks a real four-step loop against Riverside. Each iteration shows the evaluator's score bars, the one-line prompt diff the loop proposed,

and the rationale the evaluator wrote for why the change helped.

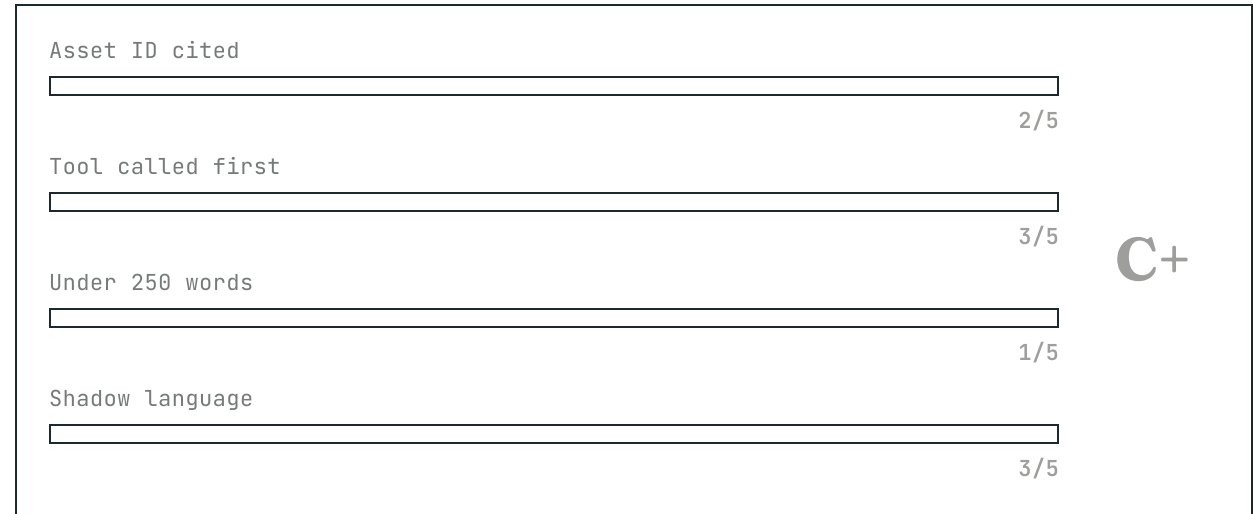
HERMES · AUTORESEARCH LOOP

EVALUATOR SCORES THE AGENT AGAINST 5 RIVERSIDE SCENARIOS; PROPOSES ONE EDIT TO HERMES.MD PER ITERATION (IDENTITY OR A SPECIFIC PLAYBOOK); KEEPS IMPROVEMENTS, DISCARDS REGRESSIONS.

V0 Baseline Grade C+	V1 Cite IDs Grade B	V2 Brevity Grade B+	V3 Playbook tune Grade A
---	--	--	---

Iteration 0 — baseline prompt

STARTING POINT, NO OPTIMIZATION



EVALUATOR NOTE

Evaluator scored the agent against the five Riverside scenarios. Two criteria miss badly: responses are too long for operator consumption, and the agent cites specific asset IDs only half the time. HERMES.md still has its generic Rung-2 framing.

AUTO-PLAY

←

→

1 / 4

Canned visualization drawn from the same evaluator pattern used in the repo. The eval harness, scoring rubric, and prompt diffs are all shipped in `/evals/` and `/hermes/agent/HERMES.md` so a reader can run the loop themselves and tune the criteria for their utility.

What the loop is doing, in one sentence: it's turning the prompt from a piece of static config into a running function of the eval set. Every scenario the agent handles badly is an opportunity for the loop to propose a tightening. Every tightening that keeps the old scores and improves the new one gets promoted into the next version. Every regression gets discarded and logged.

WHY THIS MATTERS AT A UTILITY

A protection engineer's review of an agent's recommendations is the most valuable signal in the system. The autoresearch loop is the mechanism for turning "the agent got this one wrong" into a durable improvement without anyone sitting down to rewrite the prompt by hand. It's continuous delivery for agent behavior, with the protection engineer as the evaluator.

The repo ships the evaluator (`evals/run.py`), the criteria rubric (`evals/qa_pairs.yaml`), and the prompt (`hermes/agent/prompts.py`) as three separable files. A utility that adopts this stack writes its own criteria (the ones its protection engineer cares about) and gets its own tightening loop for free.

§ 07

What a utility does with this

The honest path from cloning the repo to a Rung-3 pilot is months, not quarters, because most of the work is integration plumbing, not agent development. What the repo gives you is the scaffold.

1. **Clone and run it against SP&L.** Verify the agent behaves sensibly on data you can inspect end-to-end. This is about an hour.
2. **Swap the data adapter for your own.** Point `hermes/data/*.py` at four feeds: GIS (one-line + nameplate for one substation), OMS (the outage

archive for those feeders), historian (15-minute load and voltage for the same feeders), and the interconnection tracker (DER and microgrid inventory). Verify the same five scenarios run on your data.

3. **Wire the tool-trace stream into your SIEM.** Every tool call the agent makes emits a stable JSON record. That's the CIP-007 event-monitoring evidence, in miniature.
4. **Deploy in shadow mode on one substation.** The agent reads; it does not actuate. Log its recommendations alongside the operator's actual actions for a quarter.
5. **Review the shadow record with the protection engineer.** Where does the agent agree with existing practice? Where would it have caught something the rule-based logic missed? Where is it still worse than the operator? This is the evidence base for graduating to Rung 3.

Steps 1 and 2 are in the box. Steps 3, 4, and 5 are the consulting work, and they're the work where the real value lives for any specific utility. If that's what you want to talk about, the contact form is below.

Run Hermes at your substation.

The repo is public. The traces are committed. The notebook runs offline. Nothing you need to run this is behind a login. The Hermes runtime underneath it is open and unchanged between my office closet and a utility control room, which is the point. If you decide you want help wiring it into your operations stack, that's a separate conversation, and I'd be glad to have it.

[GITHUB REPO →](#)[JUPYTER NOTEBOOK →](#)[START A CONVERSATION →](#)

IN THIS ARTICLE

- 01 What Hermes is doing here
- 02 The substation
- 03 Five scenarios, replayed
- 04 Governance
- 05 The stack
- 06 Self-improvement
- 07 What a utility does

ARTIFACTS

- GitHub repo →
- Notebook (HTML) →
- DNM data source →

TAGS

Hermes

Agents

Open source

Gemma

Substation

VVO

Restoration

Microgrid

Autoresearch

CEII

Rung 2

NOTE

Data is synthetic. No real utility relays, real customer data, or real CEII appear anywhere in this repo. The architecture, governance pattern, and compliance gate are what a real deployment would carry forward.

Energy infrastructure software & consulting
for utilities entering the decade of the grid.

ML Playground

Get in touch

"ONE MUST IMAGINE THE OPERATOR HAPPY."

RESOURCES

SRE for Grid series

SP&L dataset

Error budget calc

Outage cost calc

CONTACT

adam@sgridworks.com

LinkedIn

GitHub

RSS (soon)